

UNIVERSITÉ DE MONTRÉAL

FACULTÉ DES ARTS ET DES SCIENCES — DÉPARTEMENT D'INFORMATIQUE

IFT 2821 — BASES DE DONNÉES

Projet de session

Gestion des inscriptions et des notes
dans une université

Édouard Bibeau	20247400
Florion-Victor Buhendwa Chalazire	20364625
Gaius Heumen Nkouantchoua	XXXXXXXXX
Imad Baida	20363685

Travail présenté à

Jihen Rezgui

16 avril 2026

Table des matières

1	Introduction et reformulation du cahier des charges	2
1.1	Reformulation	2
1.2	Justification des choix de conception	2
2	Modèle entité-association	3
2.1	Entités et attributs	3
2.2	Associations et cardinalités	4
3	Modèle relationnel	5
4	Normalisation	6
4.1	Première forme normale (1FN)	6
4.2	Deuxième forme normale (2FN)	6
4.3	Troisième forme normale (3FN)	6
5	Implantation SQL	7
5.1	LDD — création des tables	7
5.2	Requêtes SQL	7
5.3	Transact-SQL	8
6	Application web	9
6.1	Architecture et rôles	9
6.2	Fonctionnalités principales	9
6.3	Intégration des objets T-SQL	9
6.4	Captures d'écran	10
7	Conclusion	10

1 Introduction et reformulation du cahier des charges

Le but du projet est de concevoir et implanter une base de données pour gérer les inscriptions et les notes dans une université, en SQL Server. On veut couvrir tout le parcours d'un étudiant : de son admission dans un programme jusqu'à sa note finale dans un cours, en passant par l'offre de cours par semestre, les sessions d'examen et l'administration des profs.

1.1 Reformulation

L'énoncé donne six grands blocs d'information. Voici comment on les a interprétés avant de faire le modèle :

Acteurs. Il y a trois types d'utilisateurs : les étudiants, les professeurs et les administrateurs. Comme ils partagent beaucoup d'information (nom, prénom, courriel, téléphone, adresse), on a choisi de tout mettre dans une seule entité **Utilisateur** et de faire hériter les trois rôles via une relation 1-1. Ça évite d'avoir la même info dupliquée dans trois tables différentes.

Programmes et spécialisations. Chaque étudiant fait au plus un programme à la fois. Dans son programme, il peut choisir une ou plusieurs spécialisations (un peu comme une majeure/mineure), donc on a fait une association n-m avec **ChoixSpecialisation** pour garder la date et l'ordre des choix.

Catalogue de cours. Un cours a un code unique, un type (théorique, labo, TP, démo) et un nombre de crédits. Comme un cours peut appartenir à plusieurs programmes, on passe par **CoursProgramme**. Pour les prérequis on a fait une association réflexive **CoursPrerequis**, vu qu'un prérequis est lui-même un cours. Enfin, une spécialisation peut réserver des places sur certains cours, c'est géré par **Restreindre**.

Sections et horaires. L'énoncé dit qu'un cours peut être divisé en sections avec des horaires différents. On a donc séparé le cours du catalogue de son instance concrète dans un semestre : c'est la table **CoursOffert** (section, salle, horaire, capacité, mode d'enseignement).

Semestres et sessions d'examen. L'année universitaire est divisée en semestres (Hiver, Été, Automne d'une année donnée). Chaque semestre a une ou plusieurs sessions d'examen (intra, final, reprise, etc.) regroupées dans **SessionExamen**.

Évaluations et notes. Dans un cours offert on peut avoir plusieurs évaluations (TP, quiz, intra, examen final), chacune avec son poids. La note d'un étudiant à une évaluation va dans la table **Note**, qui est liée à son inscription au cours.

Inscription et historique. L'inscription d'un étudiant à un cours offert est représentée par **Inscription** : date, statut (inscrit, annulé), note finale, décision (réussi, échec, abandon), numéro de tentative et validation des prérequis.

1.2 Justification des choix de conception

- **Identifiants entiers IDENTITY** plutôt que des codes textuels : ça rend les jointures plus simples et on garde quand même un champ `code_*` lisible quand c'est utile (cours, programme).
- **Colonne calculée `mat_User`** : c'est le matricule sur 10 chiffres (AAAAMMXXXX) construit automatiquement à partir de l'année et du mois d'inscription et de l'id. Comme ça on n'a pas à le saisir à la main et il reste toujours cohérent avec les autres champs.

- **Héritage Utilisateur vers Etudiant, Professeur, Administrateur** : un même courriel ne peut pas appartenir à deux personnes, et un utilisateur peut devenir prof ou admin sans devoir recréer ses infos.
- **Cours séparé de CoursOffert** : le cours dans le catalogue et le cours offert dans un semestre donné ne sont pas la même chose. C’est ce qui permet à un étudiant de reprendre IFT2821 en hiver 2026 après l’avoir échoué en automne 2025.
- **CHECK et UNIQUE** : format du téléphone et du courriel, liste fermée pour les types de cours, d’évaluation et les grades, et surtout **UNIQUE(id_Etud, id_CoursOf)** pour empêcher la double inscription au même cours. Les lettres de notes sont aussi restreintes au barème UdeM (A+, A, A-, B+, B, B-, C+, C, C-, D+, D, E, F).

2 Modèle entité-association

Le modèle contient dix entités principales plus cinq associations qu’on a nommées parce qu’elles portent des attributs (comme la date du choix, le nombre de places, etc.).

2.1 Entités et attributs

Entité	Attributs (identifiant souligné)
Utilisateur	<u>id_User</u> , nom_User, prenom_User, dateInscriptUser, mat_User, dateNais_User, numTel_User, courriel_User, passwordHash_User, adresse_User
Etudiant (is-a Utilisateur)	<u>id_User</u> , statut_Etud, programme_Etud
Professeur (is-a Utilisateur)	<u>id_User</u> , grade_Prof, statut_Prof
Administrateur (is-a Utilisateur)	<u>id_User</u> , role_Admin_Etud
Programme	<u>id_Prog</u> , code_Prog, nom_Prog, descript_Prog, nbCredit_Prog, unite_Prog, duree_Prog, dateCreat_Prog
Specialisation	<u>id_Spec</u> , nom_Spec, descript_Spec, nbCredit_Spec
Cours	<u>id_Cours</u> , code_Cours, nom_Cours, credit_Cours, type_Cours, nbHeure_Cours
CoursOffert	<u>id_CoursOf</u> , sect_CoursOf, capacite_CoursOf, horaire_CoursOf, salle_CoursOf, dateDeb_CoursOf, dateFin_CoursOf, mondeEns_CoursOf
Semestre	<u>id_Semest</u> , nom_Semest, annee_Semest, datDeb_Semest, dateFin_Semest
SessionExamen	<u>id_SessExam</u> , type_SessExam, dateDeb, dateFin
Evaluation	<u>id_Eval</u> , type_Eval, date_Eval, nom_Eval, poids_Eval, descript_Eval
Inscription (association)	<u>id_Inscript</u> , statut_Inscript, date_Inscript, date_Desinscript, noteFi_Inscript, noteLet_Inscript, decisionFi_Inscript, tentative_Inscript, estValidePrerequis_Inscript
Note (association)	<u>id_Note</u> , valNum_Note, ValLettre_Note, retroAction, dateAttrib_Note

Entité	Attributs (identifiant souligné)
Enseigner (association)	<u>id_Enseigner</u> , nbH_Enseigner, datDeb_Enseigner, date_Fin_Enseigner
ChoixSpecialisation (association)	<u>id_ChoixSpec</u> , date_ChoixSpec, nb_ChoixSpec
Restreindre (association)	<u>id_Rest</u> , nb_Restrict

2.2 Associations et cardinalités

Association	Cardinalité	Sémantique
Etudiant — Programme	0,n — 1,1	Un étudiant appartient à au plus un programme.
Specialisation — Programme	0,n — 1,1	Une spécialisation appartient à un programme.
ChoixSpecialisation (Etudiant, Specialisation)	n — m	Un étudiant peut choisir plusieurs spécialisations.
CoursProgramme (Cours, Programme)	n — m	Un cours peut appartenir à plusieurs programmes.
CoursPrerequis (Cours, Cours)	n — m (réflexive)	Un cours peut avoir plusieurs prérequis.
Restreindre (Specialisation, Cours)	n — m	Réservation de places par spécialisation.
CoursOffert — Cours	0,n — 1,1	Une offre concrétise un cours du catalogue.
CoursOffert — Semestre	0,n — 1,1	L'offre est située dans un semestre.
Inscription (Etudiant, CoursOffert)	n — m	Unique par couple (contrainte UNIQUE).
Enseigner (Professeur, CoursOffert)	n — m	Un cours offert peut avoir plusieurs professeurs.
SessionExamen — Semestre	0,n — 1,1	Une session est rattachée à un semestre.
Evaluation — CoursOffert	0,n — 1,1	Une évaluation appartient à un cours offert.
Evaluation — SessionExamen	0,n — 1,1	Une évaluation se tient dans une session.
Note — Inscription	0,n — 1,1	Une note concerne une inscription précise.
Note — Evaluation	0,n — 1,1	Une note porte sur une évaluation précise.

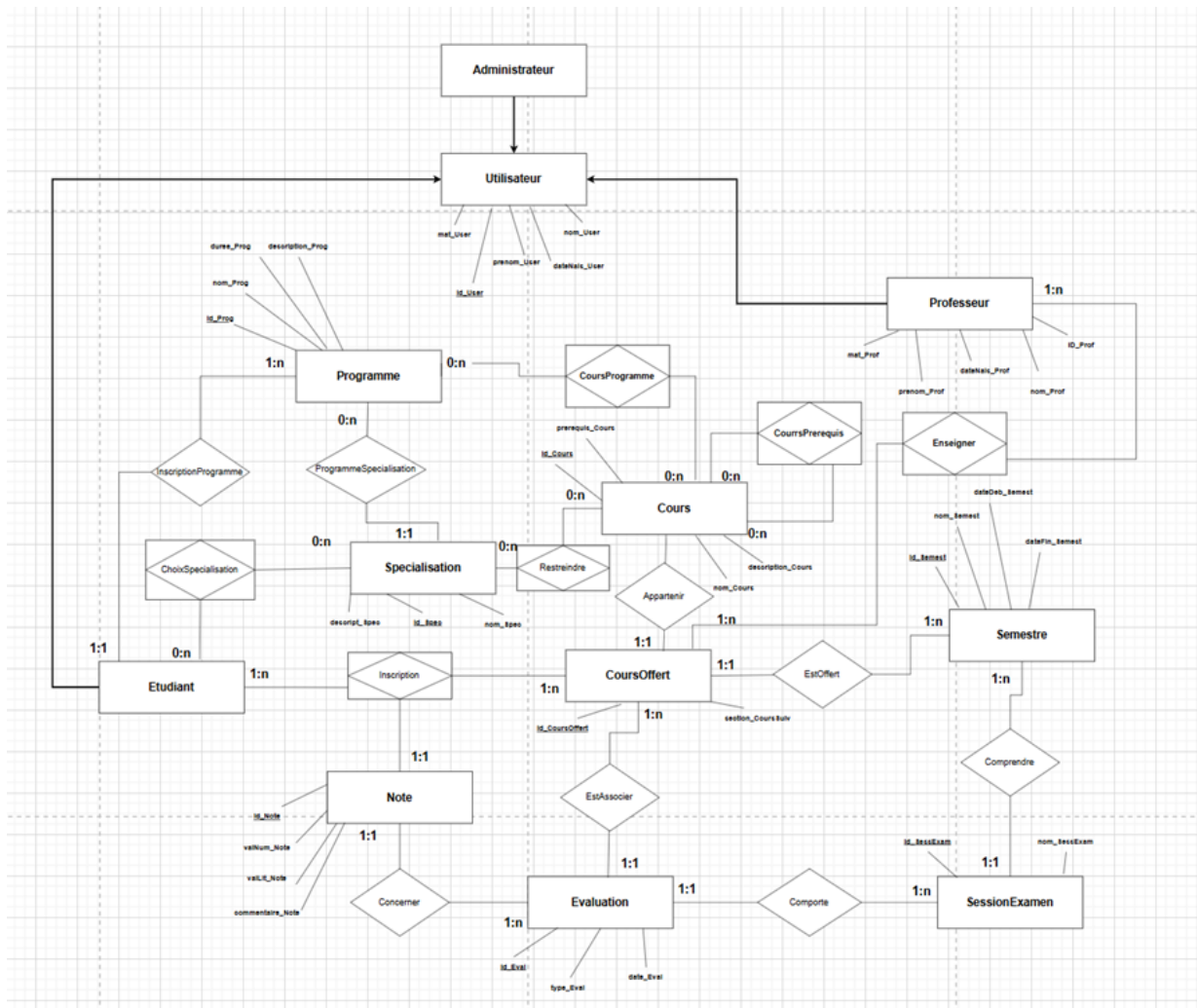


Figure 1 – Schéma entité-association de la base GestionInscriptEtudiant.

3 Modèle relationnel

La transformation du schéma E-A donne dix-sept relations. Les clés primaires sont soulignées et les clés étrangères préfixées par un #.

- Programme(id_Prog, code_Prog, nom_Prog, descript_Prog, nbCredit_Prog, unite_Prog, duree_Prog, dateCreat_Prog)
- Specialisation(id_Spec, nom_Spec, descript_Spec, nbCredit_Spec, #id_Prog_Spec)
- Utilisateur(id_User, nom_User, prenom_User, dateInscriptUser, mat_User, dateNais_User, numTel_User, courriel_User, passwordHash_User, adresse_User)
- Etudiant(#id_User, statut_Etud, #programme_Etud)
- Professeur(#id_User, grade_Prof, statut_Prof)
- Administrateur(#id_User, role_Admin_Etud)
- Cours(id_Cours, code_Cours, nom_Cours, credit_Cours, type_Cours, nbHeure_Cours)
- CoursProgramme(id_CoursProg, #id_Prog, #id_Cours)
- CoursPrerequis(id_CoursPre, #id_Cours, #id_Prerequis)
- Semestre(id_Semest, nom_Semest, annee_Semest, datDeb_Semest, dateFin_Semest)
- CoursOffert(id_CoursOf, sect_CoursOf, capacite_CoursOf, horaire_CoursOf, salle_CoursOf, dateDeb_CoursOf, dateFin_CoursOf, mondeEns_CoursOf, #id_Cours, #id_Semest)
- SessionExamen(id_SessExam, type_SessExam, dateDeb, dateFin, #id_Semest)
- Evaluation(id_Eval, type_Eval, date_Eval, nom_Eval, poids_Eval, descript_Eval, #id_CoursOf,

- #id_SessExam)
- ChoixSpecialisation(id_ChoixSpec, date_ChoixSpec, nb_ChoixSpec, #id_Etud, #id_Spec)
- Inscription(id_Inscript, statut_Inscript, date_Inscript, date_Desinscript, noteFi_Inscript, noteLet_Inscript, decisionFi_Inscript, tentative_Inscript, estValidePrerequis_Inscript, #id_Etud, #id_CoursOf)
- Note(id_Note, valNum_Note, ValLettre_Note, retroAction, dateAttrib_Note, #id_Inscript, #id_Eval)
- Restreindre(id_Rest, nb_Restrict, #id_Spec, #id_Cours)
- Enseigner(id_Enseigner, nbH_Enseigner, datDeb_Enseigner, dateFin_Enseigner, #id_Prof, #id_CoursOf)

4 Normalisation

4.1 Première forme normale (1FN)

Pour être en 1FN il faut que tous les attributs soient atomiques (pas de liste, pas d'attribut multi-valué) et que chaque ligne soit identifiable de façon unique. Dans nos tables on a tout décomposé en champs simples (l'adresse, les noms, etc.) et chaque table a sa propre clé primaire, donc la 1FN est respectée partout.

4.2 Deuxième forme normale (2FN)

Pour être en 2FN il faut déjà être en 1FN, et qu'aucun attribut non-clé ne dépende seulement d'une partie de la clé primaire. Chez nous toutes les clés primaires sont simples (un seul id auto-incrémenté), donc on ne peut pas avoir de dépendance partielle : la 2FN est respectée automatiquement.

Les principales dépendances fonctionnelles sont :

Déterminant	Déterminés
id_User	nom_User, prenom_User, dateNais_User, ..., mat_User
id_Prog	code_Prog, nom_Prog, nbCredit_Prog, ...
id_Cours	code_Cours, nom_Cours, credit_Cours, type_Cours, ...
id_CoursOf	sect, capacite, horaire, salle, #id_Cours, #id_Semest
id_Eval	type, date, nom, poids, #id_CoursOf, #id_SessExam
id_Inscript	statut, dates, noteFi, noteLet, #id_Etud, #id_CoursOf
id_Note	valNum, valLettre, retroAction, #id_Inscript, #id_Eval

4.3 Troisième forme normale (3FN)

La 3FN ajoute la condition qu'aucun attribut non-clé ne dépend d'un autre attribut non-clé (pas de dépendance transitive). En regardant nos DF ligne par ligne, toutes nos tables sont en 3FN.

Il y a par contre deux cas limites qu'on a identifiés et qu'on assume :

1. **Table Utilisateur** : $\text{id_User} \rightarrow \text{dateInscriptUser} \rightarrow \text{mat_User}$. En théorie c'est une dépendance transitive, mais **mat_User** est une colonne calculée et persistée (PERSISTED) par SQL Server à partir de la date et de l'id. Elle ne peut jamais être incohérente, donc on l'a gardée.
2. **Table Inscription** : $\text{id_Inscript} \rightarrow \text{noteFi_Inscript} \rightarrow \text{decisionFi_Inscript}$. La décision finale (Réussi, Échec, Abandon) pourrait se déduire de la note, mais on l'a stockée

explicitement pour deux raisons. D’abord, le cas *Abandon* ne se déduit pas d’une note. Ensuite, la décision peut résulter d’un processus administratif (appel, reprise) qui n’est pas juste une question de seuil. Le trigger `trg_Note_RecalculerInscription` garde quand même cette colonne synchronisée automatiquement avec la moyenne.

5 Implantation SQL

5.1 LDD — création des tables

Le fichier `database/LDD_LMD.sql` crée la base `GestionInscriptEtudiant`, les 17 tables avec leurs contraintes (PK, FK, CHECK, UNIQUE, DEFAULT), puis insère environ 190 lignes de données de test réparties sur toutes les tables.

```

1 CREATE TABLE Utilisateur (
2     id_User INT IDENTITY(100, 1),
3     nom_User VARCHAR(50) NOT NULL,
4     prenom_User VARCHAR(50) NOT NULL,
5     dateInscriptUser DATE NOT NULL DEFAULT GETDATE(),
6     mat_User AS (CAST(YEAR(dateInscriptUser) AS VARCHAR(4)) +
7         RIGHT('00' + CAST(MONTH(dateInscriptUser) AS VARCHAR(2)), 2) +
8         RIGHT('0000' + CAST(id_User AS VARCHAR(4)), 4)) PERSISTED UNIQUE,
9     dateNais_User DATE NOT NULL,
10    numTel_User VARCHAR(20) CHECK (numTel_User IS NULL OR
11        numTel_User LIKE '[1-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'),
12    courriel_User VARCHAR(100) CHECK (courriel_User LIKE '%_@_%._%'),
13    passwordHash_User VARCHAR(MAX),
14    adresse_User VARCHAR(200),
15    CONSTRAINT PK_Utilisateur PRIMARY KEY(id_User),
16    CONSTRAINT UQ_Utilisateur_courriel UNIQUE(courriel_User)
17 );

```

Listing 1 – Extrait de `LDD_LMD.sql` — table Utilisateur

5.2 Requêtes SQL

Le fichier `database/requetes.sql` contient :

- **5 requêtes simples** (R1 à R5) : lister les notes, les cours restreints par spécialisation, les affectations prof-cours, le total d’heures par professeur, et les moyennes par évaluation.
- **5 requêtes complexes** (R6 à R10), toutes avec au moins 4 tables jointes :
 - R6. Bulletin complet d’un étudiant (7 tables).
 - R7. Moyenne pondérée par étudiant et par cours (6).
 - R8. Liste des profs avec leurs cours et le programme associé (7).
 - R9. Cours restreints par spécialisation avec le programme parent (6).
 - R10. Résumé d’une session d’examen avec le prof responsable et la moyenne obtenue (8).

```

1 SELECT U.mat_User, U.nom_User, U.prenom_User,
2        C.code_Cours, C.nom_Cours,
3        ROUND(SUM(N.valNum_Note * Ev.poids_Eval) /
4            NULLIF(SUM(Ev.poids_Eval), 0), 2) AS moyennePonderee
5 FROM Note N
6 JOIN Inscription I ON N.id_Inscript = I.id_Inscript
7 JOIN Etudiant Et ON I.id_Etud = Et.id_User

```

```

8 JOIN Utilisateur U ON Et.id_User = U.id_User
9 JOIN Evaluation Ev ON N.id_Eval = Ev.id_Eval
10 JOIN CoursOffert CO ON I.id_CoursOf = CO.id_CoursOf
11 JOIN Cours C ON CO.id_Cours = C.id_Cours
12 GROUP BY U.mat_User, U.nom_User, U.prenom_User, C.code_Cours, C.nom_Cours;

```

Listing 2 – Exemple — R7 : moyenne pondérée par étudiant et par cours

5.3 Transact-SQL

Le fichier database/T-SQL.sql contient nos objets T-SQL :

Fonctions. `fn_MoyennePondereeEtudiantCours` pour calculer la moyenne pondérée d'un étudiant dans un cours donné, `fn_NoteLettre` qui applique le barème UdeM à une note sur 20, et `fn_EtudiantsAuDessusMoyenne` qui retourne une table des étudiants ayant une moyenne supérieure à un seuil.

Procédures. `sp_InscrireEtudiantCours` qui fait l'inscription en vérifiant la capacité et les prérequis dans une transaction, `sp_ReleveNotesEtudiant` qui génère le relevé de notes, et `sp_MajNoteLettreInscription` qui utilise un **curseur** pour parcourir les inscriptions et mettre à jour les lettres manquantes.

Déclencheurs. `trg_Note_RecalculerInscription` (AFTER INSERT/UPDATE/DELETE) recalcule la moyenne du cours et la lettre quand une note change, `trg_Evaluation_VerifierPoids` empêche la somme des poids des évaluations d'un cours de dépasser 100, et `trg_Inscription_EmpecherSuppressionAvecNotes` (INSTEAD OF DELETE) bloque la suppression d'une inscription qui a déjà des notes.

Vue. `v_MoyenneFinaleEtudiant` donne la moyenne générale de chaque étudiant, tous cours confondus.

```

1 CREATE TRIGGER dbo.trg_Note_RecalculerInscription
2 ON dbo.Note
3 AFTER INSERT, UPDATE, DELETE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7     DECLARE @inscriptions TABLE (id_Inscript INT PRIMARY KEY);
8     INSERT INTO @inscriptions (id_Inscript)
9     SELECT DISTINCT id_Inscript FROM inserted
10    UNION
11    SELECT DISTINCT id_Inscript FROM deleted;
12
13    UPDATE I
14    SET I.noteFi_Inscript = X.moyenne,
15        I.noteLet_Inscript = dbo.fn_NoteLettre(X.moyenne),
16        I.decisionFi_Inscript =
17            CASE WHEN X.moyenne IS NULL THEN I.decisionFi_Inscript
18                WHEN X.moyenne >= 10 THEN 'Réussi'
19                ELSE 'Échec' END
20    FROM Inscription I
21    JOIN @inscriptions A ON I.id_Inscript = A.id_Inscript
22    OUTER APPLY (
23        SELECT CAST(SUM(N.valNum_Note * Ev.poids_Eval) /
24                    NULLIF(SUM(Ev.poids_Eval), 0) AS DECIMAL(5,2)) AS moyenne
25    FROM Note N

```

```

26      JOIN Evaluation Ev ON N.id_Eval = Ev.id_Eval
27      WHERE N.id_Inscript = I.id_Inscript
28  ) AS X;
29 END;

```

Listing 3 – Extrait de T-SQL.sql — déclencheur de recalcul

6 Application web

On a fait l'application en ASP.NET Core Razor Pages (.NET 10). Pour parler avec SQL Server on utilise Entity Framework Core, et pour les mots de passe on utilise BCrypt. L'app exécute bien plus que les dix instructions SQL demandées : chaque page de dashboard, d'inscription ou de saisie de notes fait plusieurs requêtes pour charger et mettre à jour les données.

6.1 Architecture et rôles

On a trois rôles gérés par authentification par cookies : Administrateur, Professeur et Étudiant. Chaque page est protégée par une politique d'autorisation (`AdminOnly`, `ProfesseurOnly`, `EtudiantOnly`) via l'attribut `[Authorize]`.

6.2 Fonctionnalités principales

- **Authentification.** Inscription, connexion, déconnexion, hachage BCrypt des mots de passe, et attribution automatique du rôle au moment de la connexion.
- **Étudiant.** Tableau de bord avec le dossier, les choix de spécialisation et les notes par cours. Page d'inscription qui filtre les cours selon le programme et vérifie la capacité, plus la possibilité de se désinscrire.
- **Professeur.** Liste des cours enseignés avec les étudiants inscrits, et un petit formulaire pour saisir la note finale. La note se rentre entre 0 et 20, la lettre est calculée automatiquement en JavaScript pendant la saisie, et la base valide le tout avec le trigger et le CHECK.
- **Administrateur.** Dashboard avec des stats (nombres d'étudiants, profs, cours, inscriptions), gestion des rôles et affectation des profs aux cours offerts.

6.3 Intégration des objets T-SQL

Plutôt que de tout faire en LINQ avec EF Core, on a branché quelques endroits critiques directement sur nos objets T-SQL pour laisser le SGBD faire son travail.

sp_InscrireEtudiantCours. On l'appelle dans la page d'inscription de l'étudiant (`Pages/Etudiant/Inscription.cshtml.cs`) chaque fois qu'un nouvel étudiant s'inscrit à un cours. La procédure vérifie la capacité, la non-duplication et les prérequis, le tout dans une transaction. Si la capacité est dépassée, le `RAISERROR` remonte jusqu'au code C# qui attrape la `SqlException` et affiche le message à l'étudiant.

v_MoyenneFinaleEtudiant. On lit cette vue dans le dashboard admin (`AdminDashboard.cshtml.cs`) avec `SqlQueryRaw<MoyenneEtudiantVue>` pour afficher le top 10 des moyennes.

Déploiement auto. Au démarrage, `Program.cs` lit `T-SQL.sql` et l'exécute batch par batch. Comme le script utilise des `DROP IF EXISTS` avant chaque `CREATE`, il est idempotent, donc on n'a pas à y penser entre deux runs de l'application.

6.4 Captures d'écran

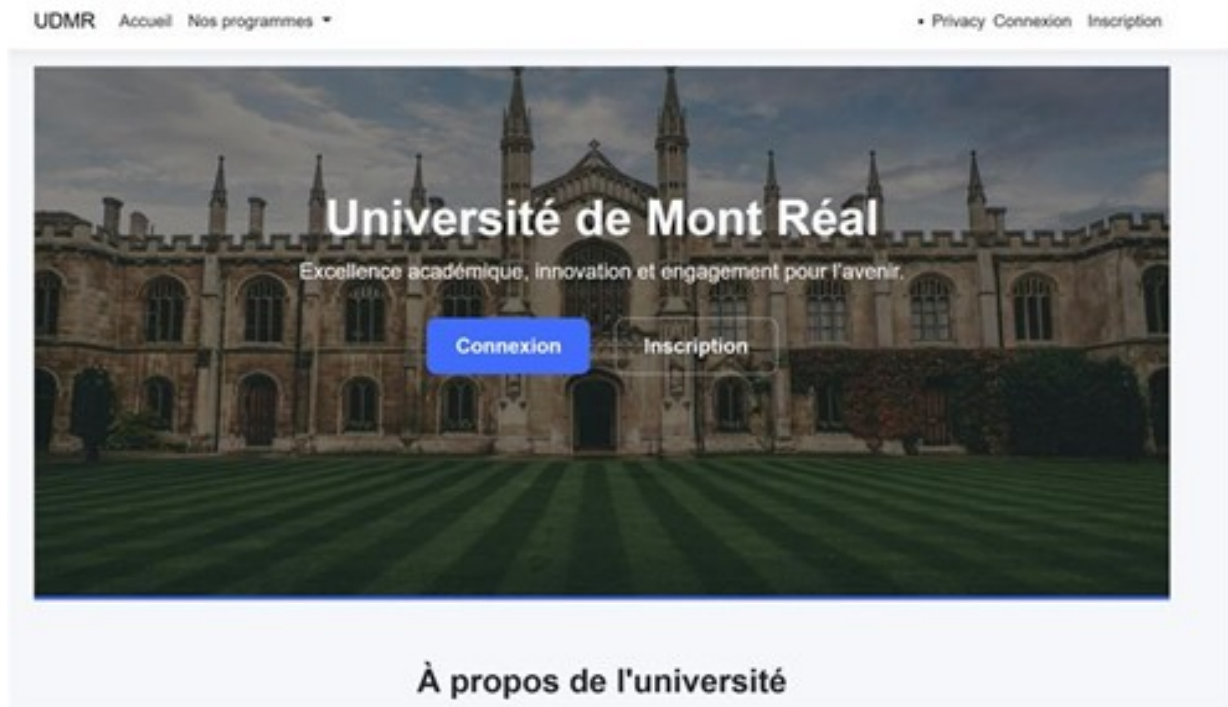


Figure 2 – Page d'accueil et formulaire de connexion.

7 Conclusion

Dans ce projet on a couvert toutes les étapes demandées : le modèle E-A avec 17 relations toutes en 3FN, la base SQL Server avec environ 190 tuples de test, 10 requêtes SQL (5 simples et 5 complexes à 4 tables ou plus), nos objets T-SQL (3 triggers, 3 fonctions, 3 procédures dont une avec un curseur, et une vue) et finalement une application web en ASP.NET Core pour les trois rôles. En plus des contraintes déclaratives (PK, FK, CHECK, UNIQUE), on utilise les triggers pour garder la base cohérente dynamiquement : recalcul automatique des moyennes, somme des poids d'évaluations qui ne peut pas dépasser 100, et protection contre la suppression d'une inscription qui aurait déjà des notes.